Project Summary — Healthcare SQL Analytics

Use this as your working notes file, and as source material for your LinkedIn post. Fill in the blanks with your actual query results.

What This Project Demonstrates

End-to-end SQL analytics on a real-world-style dataset: taking a raw CSV with no structure guarantees, designing a proper relational

schema for it, cleaning it, and extracting business-relevant insights
using progressively more advanced SQL techniques — aggregation, date functions, and window functions — finishing with reusable views that mimic a dashboard data layer.

Process Log

1. **Schema design** — created the `patients` table with appropriate data types for each field (VARCHAR for text, DATE for dates,
   NUMERIC(12,2) for currency).
2. **Data import** — loaded 55,500 records via pgAdmin's

Import/Export
tool. Diagnosed and fixed an import error caused by `patient_id`
being mapped to the CSV's `name` column; solved by excluding `patient_id` from the import's column list so it auto-generates.

3. **Exploration** — checked row counts, distinct categories (medical
conditions, admission types), null values, and duplicate records.
4. **Cleaning** — standardized text casing, validated that discharge dates always follow admission dates, and added a derived `length_of_stay` column.

5. **Aggregation** — computed case counts, average/total billing, and hospital-level performance metrics using `GROUP BY`.
6. **Time analysis** — analyzed admission trends by month, day of
   week, and year using `DATE_TRUNC`, `TO_CHAR`, and `EXTRACT`.
7. **Window functions** — ranked patients by billing within each hospital, computed a running total of billing over time, and calculated each condition's percentage share of total billing.
8. **Views** — packaged the condition- and hospital-level summaries

into `vw_condition_summary` and `vw_hospital_summary` for reuse.

Key Numbers

- Total records: **55,500**
- Most common condition: ___ (___ cases)
- Highest average billing condition: ___ ($___)
- Busiest hospital: ___ (___ patients)
- Longest average stay: ___ (___ days)
- Peak admission month: ___
- Top insurance provider: ___

Reflections / Challenges

- Import errors are easy to misdiagnose from the summary popup alone — the real error only surfaces in pgAdmin's Processes → process log, which showed the exact PostgreSQL error text.
- Synthetic Kaggle datasets can contain incidental duplicate-looking rows (same name/date/hospital) that are a data-generation artifact, not a pipeline bug — worth checking total row count against the dataset's documented size before assuming an import error.

LinkedIn Post Draft

> I analyzed 55,500 hospital admission records using PostgreSQL and pgAdmin to uncover cost, admission, and hospital performance patterns.
>
> Highlights:
>
> - ___ was the most common condition, at ___ cases
> - ___ had the highest average treatment cost ($___)
> - Hospital ___ handled the highest patient volume
>
> Built with PostgreSQL, pgAdmin, and SQL — covering schema

> design,
> data cleaning, aggregation, time-series analysis, and window functions.
>
> Full project + schema on GitHub: [link]
>
> #SQL #PostgreSQL #DataAnalytics #HealthcareAnalytics